TrustHome PH

Functional Specification & Business Rules

The business rules and step-by-step behaviour of every TrustHome feature as it works today, with known gaps and the best practices for changing each one.

Document	TH-03 · Functional Specification & Business Rules
Version / date	Rev. 2.0 · 4 October 2026
Factual baseline	TrustHome_Current_Progress_Report_2026-10-04.md (PR #42 `fixes` working copy, based on `features/v1-3-navigation-settings`; excludes `dev`)
Replaces	Build Instructions §2–6 + System Build Plan §3–8
Audience	Team Idle developers, Scrum Master, QA, panel/adviser reviewers, and AI coding assistants used by the team

Evidence label	Meaning (never use these interchangeably)
[Implemented in source]	Code exists in the repository working copy. Not proof that it runs in production.
[Unit/emulator tested]	Covered by automated unit tests or the Firestore emulator suite.
[Reported working]	A team member observed it working in the browser against the test project.
[Deployed/live verified]	Deployed to trusthome-ph and confirmed working there.
[Known gap]	Does not yet meet the product claim. Do not present it as working.
[Roadmap] / **[Proposed]**	Planned future work / a recommendation in this document, not yet built.

Contents

1. How to Use This Specification . . . 3
2. Accounts, Roles and Admin Access . . . 3
3. Listing Intake and Verification . . . 4
3.1 Listing fields and validation (client: `lib/listingValidation.js`; mirrored by rules) . . . 4
3.2 Submission flow (current) . . . 4
3.3 Admin review flow . . . 4
3.4 Approximate map and exact-address privacy . . . 5
4. Booking Workflow . . . 5
4.1 State machine . . . 5
4.2 Overlap rule . . . 6
4.3 Approval as implemented today [[user]] [[test]] . . . 6
4.4 Completion . . . 7
4.5 Deposit accountability . . . 7
4.6 Recommended fixes for the booking gaps [[prop]] . . . 7
5. Ratings . . . 8
6. Disputes and Public Accountability . . . 8
7. Trust & Fairness Score . . . 8
7.1 Formula . . . 8
7.2 Comparable listings . . . 8
7.3 Anti-gaming . . . 9
7.4 Where it runs . . . 9
8. Notifications . . . 9
9. Bank-Acquired Catalog and Loan Estimate . . . 10
9.1 Loan estimate . . . 10
10. Dashboards, My Activity and Settings . . . 10
11. Requirement Traceability Summary . . . 11

1. How to Use This Specification

Each feature below has: what it does, the exact business rules, the step-by-step flow as it works **today**, the evidence label, the known gaps, and the best practices to follow when changing it. Functional requirement IDs (FR-xxx) and use-case IDs (UC-xx) match the Use Cases & Test Cases workbook.

Feature	FR	Owner	Evidence
Accounts, roles, admin access	FR-001, FR-019	Andresa	[Implemented in source] [Unit/emulator tested]
Listing intake & edit	FR-002, FR-003	Carillo	[Reported working] [Unit/emulator tested]
Admin verification review	FR-003	Carillo / Andresa	[Reported working]; [Known gap] PDF delivery
Approximate map & private address	FR-022, FR-024	Carillo	[Implemented in source]; [Known gap] address handoff
Search & browse	FR-004	Laurente	[Implemented in source]
Booking request / approval	FR-005, FR-006	Laurente	[Reported working] [Unit/emulator tested]; [Known gap] race
Deposit reference, completion	FR-007, FR-008	Laurente	[Implemented in source]
Ratings	FR-008	Prades	[Unit/emulator tested]
Disputes & accountability	FR-009, FR-018	Laurente	[Unit/emulator tested]
Trust & Fairness Score, anti-gaming	FR-010, FR-011	Prades	[Unit/emulator tested]
Notifications	FR-013, FR-020	Andresa	[Implemented in source]
Bank catalog & loan estimate	FR-014, FR-015	Solpico	[Unit/emulator tested]
Dashboards, My Activity, Settings	FR-012, FR-016, FR-021, FR-023	Carillo / Andresa	[Implemented in source]

2. Accounts, Roles and Admin Access

- Sign-in methods: Firebase email/password and Google. One `users/{uid}` profile per UID, created on first sign-in.
- `AuthContext` subscribes to `users/{uid}` and exposes `profile` and `role`. Incomplete legacy profiles are normalised to an active regular user **without overwriting an existing role**.
- `RequireAuth` protects signed-in pages and returns to the requested page after login.
- `RequireAdmin` renders admin routes only for `role == admin` **and** `status == active`. While loading it shows a visible status; if the profile is missing or unreadable it **fails closed** with a retry message.
- Suspended users see a suspended-account notice; rules block their writes (list, book, rate, dispute).
- Users can edit allowed profile fields but can never change their own `role` or `status`.

BEST PRACTICE · Authentication & authorization

- Authorize in Firestore rules; treat route guards as navigation only.
- Never grant admin by email address in client code. Promote admins by writing `role: admin` with the Admin SDK or from an existing admin account, and log who did it.
- Keep the admin count small and use separate admin test accounts for QA.
- Show the same generic error for “wrong password” and “unknown email” to avoid account enumeration.

3. Listing Intake and Verification

3.1 Listing fields and validation (client: `lib/listingValidation.js`; mirrored by rules)

Field	Rule	Stored in
Title	Required	listings
Property type	Required (room, studio, apartment, house, condo, etc.)	listings
Purpose	Required: **rent** or **sale**. Sale listings are not bookable and are excluded from rental fairness comparisons	listings
Rental term / price period	Required for rent purpose	listings
Exact address	Required; **never** written to the public listing	listingPrivate
City (and area)	Required; used for browse filters and fairness comparables	listings
Price (PHP)	Required, positive number	listings
Rooms, bathrooms, floor area (m²), lot area	Required where applicable; positive numbers. Floor area drives price-per-m² fairness	listings
Availability date	Required	listings
Amenities	Multi-select	listings
Description	Required, non-blank, **max 400 words**. 150–400 words is a recommendation shown in the UI, not a minimum	listings
Photos	At least **4** images; recommend 8–15 at a consistent 4:3 ratio	listings.photoUrls (Cloudinary)
Ownership document	Exactly one required (title/TCT, tax declaration/receipt; supporting: mortgage statement, insurance)	listingPrivate
Government photo ID	Required; name must match ownership document	listingPrivate
Showing windows	At least one valid window (day + start < end)	listings
Map pin	Optional; stored only as a 5-character geohash with an approximate flag	listings

3.2 Submission flow (current)

1. Owner completes the multi-step form; client validation blocks submit and shows field-level errors.
2. Photos and documents upload to Cloudinary via the unsigned `trusthome_uploads` preset. PDFs use resource type `raw`; images use `auto/image`. Assets are placed in listing-specific folders with tags.
3. A single **Firestore batch** writes `listings/{id}` (public fields, `verificationStatus: pending`, photo URLs) and `listingPrivate/{id}` (exact address, document URLs). Both succeed or neither does.
4. A best-effort admin notification (`recipientId: "__admins__"`) links to the review item. Failure to notify does not fail the submission.
5. The listing is invisible to the public until approved. The owner sees it in My Listings as pending.

3.3 Admin review flow

1. Admin opens `/admin/listings` (query: pending). Photos, ownership document and government ID are shown beside the account name, with a **Download** link for documents.
2. Plausibility check: does the ID name match the ownership document and the account? Is the document complete, legible, and not obviously altered? Do the photos match the described property?

3. Decision: **Approve** (status verified, `verifiedAt` set, public with badge "Verified – document reviewed"), **Request changes** (note shown to owner; owner edits and resubmits → pending), or **Reject** (note shown; not public).
4. Owner receives an in-app notification for each decision.

KNOWN GAP · Verification-document delivery (P0)

- Cloudinary currently refuses to deliver the uploaded PDF (`show_original_customer_untrusted`). The admin Download link adds the attachment flag but cannot override an account-level block.
- Document URLs from the unsigned preset are public if known; Firestore privacy does not protect the file itself.
- Do not run `migrate_verification_documents.py --apply` yet — the app still reads `ownershipDocumentUrl` / `governmentIdUrl` and cannot yet consume authenticated asset metadata. See TH-04 §6 and TH-06 §5.

BEST PRACTICE · Verification best practices

- Write the plausibility limitation in the UI (create page and listing badge tooltip) — never imply a legal title check.
- Use a reviewer checklist and record who approved and when; keep the review note factual and actionable.
- Data minimisation: request one ownership document and one ID only; advise owners to watermark copies "For TrustHome verification only".
- Define a retention period for documents after review and an owner-initiated deletion path (Data Privacy Act of 2012 principles: proportionality, purpose limitation).
- Edits to a verified listing's critical fields (address, documents, price) should return it to review; cosmetic edits need not.

3.4 Approximate map and exact-address privacy

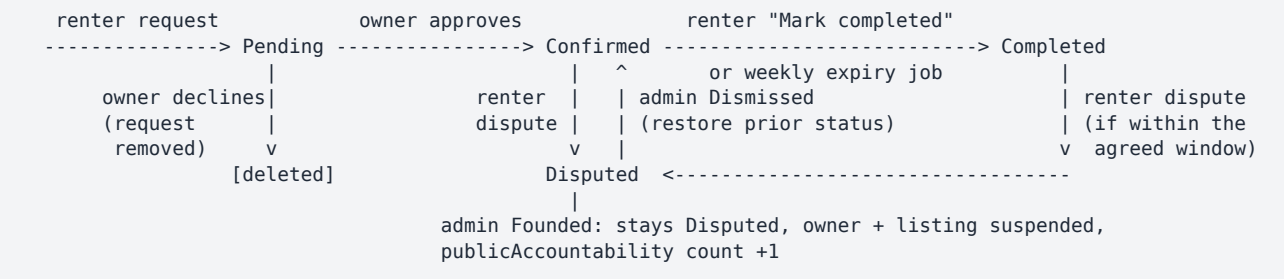
- Owner may search a location (geocoder with Philippines-specific retry) or use the private address to place a pin. Only a **5-character geohash** (roughly a few-kilometre cell) and an approximate flag reach the public listing; no latitude/longitude, no address.
- Listings without a pin hide the map section.
- The exact address stays in `listingPrivate`, readable by the active owner and admins.
- Intended rule: the renter sees the exact address only after a booking is **Confirmed** (FR-024).

KNOWN GAP · Address handoff not active

The current client approval updates booking status and timestamps only; it does **not** copy `listingPrivate.address` onto the booking. The callable in `functions/index.js` does copy it, but that path is not deployed. Until fixed, owners must share the address directly and the UI must not promise automatic disclosure.

4. Booking Workflow

4.1 State machine



Transition	Who	Preconditions	Side effects
create → Pending	Renter	Signed in, active; listing verified, purpose rent; not own listing; valid future date range	Owner notified (best effort)
Pending → Confirmed	Listing owner only	Request still exists and is still Pending; no overlap with Confirmed bookings (client check)	Renter notified; dates become unavailable
Pending → removed	Listing owner	Status Pending	Request disappears; no "Cancelled" status is written
Confirmed: add deposit reference	Renter	Status Confirmed; non-empty text	Visible to renter and owner
Confirmed → Completed	Renter (only manual authority) or trusted expiry job	Status Confirmed	Rating prompt; counts toward trust score
Confirmed/Completed → Disputed	Renter	No open dispute on booking; eligible status	Dispute + booking update in one batch; prior status saved; admins notified
Disputed → prior status	Admin (Dismissed)	Open dispute	No flag; reporter notified
Disputed stays	Admin (Founded)	Open dispute	Owner account + listing suspended; accountability +1

4.2 Overlap rule

Two date ranges overlap when `existing.start < requested.end && existing.end > requested.start`. Touching edges (check-out day equals next check-in day) do **not** overlap. Only **Confirmed** bookings block dates; multiple Pending requests for the same dates are allowed as competing requests.

```
// lib/booking.js - pure and unit-tested
export function rangesOverlap(aStart, aEnd, bStart, bEnd) {
  return aStart < bEnd && aEnd > bStart;     // half-open intervals [start, end)
}
```

4.3 Approval as implemented today [Reported working] [Unit/emulator tested]

1. Owner opens Booking Requests (or My Activity → Booking Requests) and sees a day-number availability grid.
2. Client queries Confirmed bookings scoped to the owner (`ownerId == uid`) for that listing and checks overlap.
3. A Firestore **transaction** re-reads the request and verifies it still exists, belongs to this owner, and is still Pending, then sets `status: Confirmed` and timestamps.
4. Rules allow only the owner to perform this narrow Pending → Confirmed update (changed keys limited).
5. Confirmation notification to the renter is best effort.

> **KNOWN GAP · Race condition**
> The overlap query in step 2 happens **outside** the transaction, and web-SDK transactions cannot run queries. Two approvals executed at the same moment for overlapping dates can both pass. Do not claim atomic double-booking prevention until one of the fixes below is in place and tested with concurrent approvals.

4.4 Completion

- The renter is the single manual authority for “Mark completed” (the owner button was removed to prevent duplicate completion).
- `complete_expired_bookings.py` completes Confirmed bookings whose end date has passed; GitHub Actions runs it **weekly** before trust-score recomputation. A daily Cloud Function exists in source but is not deployed.

4.5 Deposit accountability

Renter and owner settle the deposit directly (GCash, bank transfer). After confirmation the renter stores the **transaction reference** on the booking. This is a text record only — TrustHome never collects, holds or refunds money.

BEST PRACTICE · Deposit copy and data

- Label the field “Transaction reference (e.g. GCash ref. no.)” and warn users never to enter PINs, OTPs or full account numbers.
- Advise renters to pay only after confirmation and to keep their own receipt; state clearly that TrustHome cannot reverse payments.

4.6 Recommended fixes for the booking gaps [Proposed]

Option	How	Pros	Cons
A. Deploy the callable	Upgrade to Blaze, deploy `functions/index.js` confirm callable (per-listing lock, copies address). UI calls it instead of the client transaction	Strongest guarantee; server reads listingPrivate	Requires billing account; Cloud Build; cold starts
B. Client transaction on a per-listing availability document	Keep `availability/{listingId}` with an array of confirmed ranges. In one transaction: read it + the request, check overlap against the array, append the range, set booking Confirmed, and copy the address read from `listingPrivate` (owner can read it). Rules use `getAfter()` to require the availability doc to change in the same commit and allow the `address` field only on Pending → Confirmed by the owner	Works on the free plan; transaction retries on concurrent writes, so the second approval sees the first range	Rules become more complex; array must be pruned of past ranges
C. Narrow the claim	Keep current code; describe it as “owner-checked availability” and document the race in the defense	No work	Does not meet the original “impossible to double-book” claim

Whichever option is chosen: add an emulator test that fires two approvals concurrently and asserts exactly one succeeds, and a test that the renter can read the address only after confirmation.

5. Ratings

- Only an eligible renter on a **Completed** booking may create a rating; score must be an integer 1–5, optional comment.
- One rating per rater per booking; duplicates are blocked.
- Ratings are public; admins can moderate.
- New ratings affect the trust score at the next recomputation run.

> **BEST PRACTICE · Review integrity**
> Tie every rating to a completed booking (no free-form reviews), show the review date, and give admins a moderation action with a reason. Never let owners edit or delete ratings about their own listings.

6. Disputes and Public Accountability

1. Renter opens an eligible booking and submits a reason (required) and evidence description.
2. One **batch** creates `disputes/{id}` (status Open) and updates the booking to Disputed, storing the prior status and the dispute link. A second open dispute on the same booking is blocked.
3. Admins are notified through the `__admins__` feed.
4. Admin reviews booking, reporter, reason and deposit reference, then marks **Founded** or **Dismissed** with notes. Resolver and `resolvedAt` are recorded.
5. Founded: owner account and listing suspended, booking stays Disputed, `publicAccountability/{ownerUid}` +1 (shown on the public profile). Dismissed: booking restored to its prior status, no flag. Reporter notified either way.

> **CAUTION · Policy decision pending**
> Whether disputes are allowed on **Completed** bookings, and for how many days after completion, is still to be agreed (test TC-DISP-009). Record the decision here and in rules.

> **BEST PRACTICE · Fair process**
> - Give the owner a way to respond before a Founded decision (at minimum an admin note recording that the owner was contacted).
> - Use neutral public wording (“1 founded dispute”) — never publish the reporter’s identity or evidence.
> - Provide an appeal/reinstatement path in User Management and log every suspension with a reason.

7. Trust & Fairness Score

7.1 Formula

```
trust = 0.40 * booking_score + 0.40 * rating_score + 0.20 * fairness_score     (each normalised 0..1)
display = trust * 5   -> shown with a plain-language cue, e.g. "4.6 - Highly rated"

booking_score : completed bookings only, recent ones weigh more (documented baseline: within 6 months = 1.0,
                older = 0.5), normalised in lib/trustScore.js
rating_score  : average rating / 5 ; no ratings -> defined default, never NaN
fairness      : price per m2 vs median of comparables -> Above market / At market / Below market / Insufficient data
                (baseline thresholds: more than +15% = Above, below -15% = Below, otherwise At)
```

7.2 Comparable listings

- Verified **rental** listings only, same city, same property type, same price period.
- Floor area within ±20% of the subject listing.

- Median of price per m² (computed in code — Firestore has no median).
- Too few comparables → label “Insufficient data”. Sale listings are excluded entirely.

7.3 Anti-gaming

- Only Completed bookings count; Pending, declined and disputed requests never add score.
- Three completed bookings between the same renter–owner pair within 30 days set a manual-review flag (`manualReviewRequired`, pattern count) for admins.

7.4 Where it runs

The **Python job** `bank_scraper/trust_scores.py` recomputes and stores `trustScores/{listingId}` (weekly via GitHub Actions, after expired-booking completion). The app **reads the cache**; `src/lib/trustScore.js` holds the same formula for display/tests. The Cloud Function version in the old plan is not deployed.

> **BEST PRACTICE · Score maintenance**
> - Keep one shared JSON fixture of listings/bookings/ratings with expected outputs and test both `trustScore.js` and `trust_scores.py` against it (parity test).
> - Store `computedAt` and show “Score updated <date>”; a stale cache should be visible, not hidden.
> - Change weights only with a documented reason and re-run the parity fixture.
> - Explain the score in one sentence on the listing page; a bare number means nothing to a new visitor.

8. Notifications

Event	Recipient	Notes
New listing submitted	`__admins__`	Links to review item
Listing approved / changes requested / rejected	Owner	Includes review note
Booking requested	Listing owner	Never broadcast to admins
Booking confirmed	Renter	
Dispute submitted	`__admins__`	
Dispute resolved	Reporter	

- Notifications are Firestore documents; feeds are scoped by `recipientId`. Admin pages read the `__admins__` feed; admins’ own bookings still go to their personal feed.
- Writes are **best effort**: the main action succeeds even if the notification write fails (the failure is logged).
- Rules restrict which types may target `__admins__` and who may read them.

9. Bank-Acquired Catalog and Loan Estimate

- Separate from owner listings: reads active `bankProperties` documents; never bookable.
- Filters: bank, city, keyword (lowercase prefix search), price range; sorting; pagination; result count; source link to the bank page; “last seen” date.
- The UI currently exposes **Metrobank only**. Landbank and BDO are not active (TH-06).
- Admins can queue a refresh at `/admin/bank-catalog` (creates a `scraperJobs` document processed by a trusted worker).

9.1 Loan estimate

```
P = price - down_payment            r = annual_rate / 12            n = years * 12
monthly = P * r * (1+r)^n / ((1+r)^n - 1)          if r == 0: monthly = P / n
total_interest = monthly * n - P

Check: price 2,000,000, 20% down, 6%, 20 years -> P = 1,600,000 -> monthly approx. PHP 11,462.90
```

> **RULE · Required wording**
> Show “Estimate only — not a loan offer or approval. Actual rates, fees and eligibility are set by the bank.” Validate inputs (no negative values, down payment below 100%, term > 0) so the result is never NaN or Infinity.

10. Dashboards, My Activity and Settings

- **My Dashboard** (`/dashboard`): owner metrics and recent activity; counts must match Firestore and show empty states for new users.
- **My Listings**: card per listing with photo, city, type, bedrooms, price, verification status, review note, pending-request count (shows “unavailable”, not 0, if the count fails to load). No address or documents.
- **My Activity** (`/my-activity`): tabs for listings, bookings and booking requests; tab state is in the URL. Older direct routes still work.
- **Admin dashboard**: pending listings, open disputes, recent activity, with error + retry.
- **User management**: suspend/reinstate with a danger-styled button and confirmation.
- **Settings** (`/settings`, `/admin/settings`): System/Light/Dark appearance; Larger text, High contrast, Reduce motion. Stored per device (local preference).

11. Requirement Traceability Summary

FR	Requirement (current wording)	Status
FR-001	Users register/sign in with email or Google; one profile per UID	[Implemented in source]
FR-002	Listing requires core fields, ≥4 photos, showing window; description non-blank ≤400 words	[Unit/emulator tested] [Reported working]
FR-003	Ownership doc + gov ID; admin approve / request changes / reject; verified-only publication	[Reported working]; [Known gap] doc delivery
FR-004	Browse verified listings with filters and trust sort	[Implemented in source]
FR-005	Renter requests booking on verified rental listing; owner approves/declines	[Reported working]
FR-006	Confirmed ranges cannot overlap	[Implemented in source]; [Known gap] concurrency
FR-007	Renter logs deposit reference on Confirmed booking	[Implemented in source]
FR-008	Renter completes; expiry job completes; ratings after completion	[Implemented in source] [Unit/emulator tested]
FR-009	Dispute flow with Founded/Dismissed and status restore	[Unit/emulator tested]
FR-010 / 011	Trust & Fairness Score; self-booking flag	[Unit/emulator tested]
FR-012 / 016 / 023	Owner dashboard; admin dashboard & user management; My Listings	[Implemented in source]
FR-013 / 020	In-app notifications; admin feed limited to listing submissions and disputes	[Implemented in source]
FR-014 / 015	Bank catalog (Metrobank) and loan estimate	[Unit/emulator tested]
FR-017	Saved searches — retired (redirect)	Retired
FR-018	Public profile with founded-dispute count	[Implemented in source]
FR-019	Fail-closed admin access	[Unit/emulator tested]
FR-021	My Activity hub and Settings	[Implemented in source]
FR-022	Approximate geohash pin; exact address private	[Implemented in source]
FR-024	Exact address visible to renter after confirmation	[Known gap]